

# Software fault isolation for micro-eBPF

Software Security: Mini-project 2

Students: Marcus Friis-Hansen & Anders Hoeck & Rasmus Voss

UNI-ID: lns611 & lpz950 & khp836

October 7, 2026

# Contents

|          |                                                  |          |
|----------|--------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                              | <b>3</b> |
| <b>2</b> | <b>Baseline</b>                                  | <b>3</b> |
| 2.0.1    | Design . . . . .                                 | 3        |
| 2.0.2    | Implementation . . . . .                         | 3        |
| 2.0.3    | Evaluation . . . . .                             | 4        |
| <b>3</b> | <b>Extensions &amp; Variations</b>               | <b>4</b> |
| 3.1      | Chosen implementation: Address Masking . . . . . | 4        |
| 3.1.1    | Design . . . . .                                 | 4        |
| 3.1.2    | Implementation . . . . .                         | 5        |
| 3.1.3    | Evaluation . . . . .                             | 7        |
| 3.2      | Extensions . . . . .                             | 7        |
| 3.3      | Other possible variations . . . . .              | 9        |

# 1 Introduction

The purpose of the project was to produce a rewriter, that when given a micro-BPF program, transforms it, such that it never accesses memory outside its given data region, and never transfers control to an address outside its code. Otherwise the original program  $P$  is equivalent to the rewritten version  $P'$ .

## 2 Baseline

### 2.0.1 Design

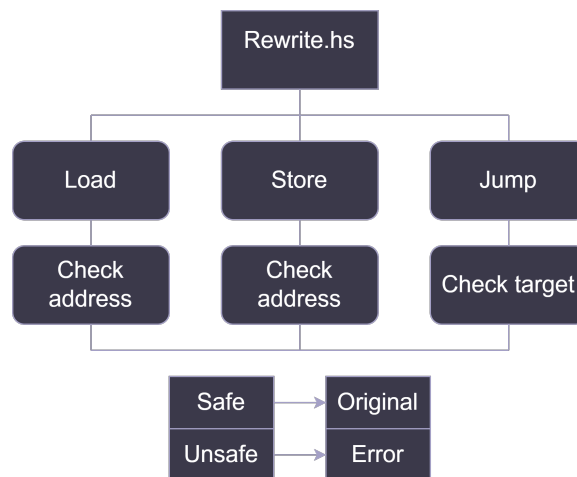


Figure 1: Our SFI design

The basic idea for a baseline, shown in figure 1 is a rewriter that takes an existing program ( $p$ ) and transforms it into another program ( $p'$ ) which satisfies the two main rules of the SFI policy.

1. Memory access must stay inside  $[DB, DL)$
2. Control flow must stay inside the program

Memory safety is enforced by checking all **Load** and **Store** instructions, while control-flow safety is enforced by checking the targets of jumps.

### 2.0.2 Implementation

For our actual implementation we mainly modified  
`rewrite :: Program -> Either String Program`

This function takes the original program and constructs a new program where the necessary safety checks have been inserted.

We add a small prologue that stores the boundaries of the allowed data region in scratch registers.

```
Binary B32 Mov (Reg 11) (R (Reg 1))
Binary B32 Mov (Reg 12) (R (Reg 10))
```

here, `r11` contains (DB) and `r12` contains (DL). We then use `r13` as a temporary register when checking memory addresses.

The main part of the rewriter is the `transform` function. It examines each instruction and decides whether additional instructions are needed. For a `Load` or `Store`, we first calculate the effective address in `r13`.

For control flow, the rewriter handles jumps separately. Jumps that already have a valid target inside the program are preserved. If a jump has a target outside the program, it is redirected to a special `ErrorStub` instead.

Finally, after all instructions have been transformed, the new program is passed to `assemble`. This produces the final program with the correct instruction offsets and jump targets.

### 2.0.3 Evaluation

For evaluation, we tested the implementation on all example programs provided in `examples/*.asm`. In total, there were 24 different example programs. For each program, we first rewrote it using our implementation and then compared the original and rewritten programs. For each example, we tested 1000 different initial states. The tests check that the rewritten program preserves the behavior of the original program, including its final result and memory state. It also checks that the rewritten program does not end in an SFI violation. All 24 example programs passed all 1000 test cases with 0 failed tests and 0 inconclusive tests.

We also tested examples specifically targeting unsafe behavior. For example, `wild_pointer.asm` resulted in an SFI violation for 771 out of 1000 initial states in the original program, but no violations after the rewrite.

Overall, the testing provides evidence that our implementation preserves the tested behavior of the original programs while enforcing the required SFI restrictions. Since the evaluation is based on a finite set of example programs and randomly generated initial states, it does not by itself mean a formal proof of correctness.

## 3 Extensions & Variations

### 3.1 Chosen implementation: Address Masking

#### 3.1.1 Design

For our chosen variation, we implemented address masking instead of the bounds checks used in the baseline. Rather than checking whether each memory address is inside the data region and going to the error stub if it is not, address masking changes the address so that it points inside the allowed region.<sup>1</sup>

---

<sup>1</sup>Gang Tan, *Principles and Implementation Techniques of Software-Based Fault Isolation*, Section 3.1.3, pp. 152–153.

Let

$$size = DL - DB.$$

For our masking approach to work, the size of the data region must be a power of two,

$$size = 2^m,$$

where  $m \geq 9$ . The reason is that when the size is a power of two,

$$size - 1 = 2^m - 1$$

has exactly its lowest  $m$  bits set to 1. Therefore,

$$EA \& (size - 1)$$

keeps only the lowest  $m$  bits of the effective address. The result is always between 0 and  $size - 1$ . In other words,

$$EA \& (size - 1) = EA \bmod size.$$

We also require DB to be aligned to the region size:

$$DB \bmod 2^m = 0.$$

This means that the lowest  $m$  bits of DB are zero. For an address that is already inside the data region, masking therefore gives its offset from DB. Adding DB again gives the original address. So valid addresses remain unchanged.<sup>2</sup>

We create the mask as

$$mask = size - 1$$

and calculate the safe address as

$$safeAddr = DB + (EA \& mask).$$

Since the masked part is always between 0 and  $size - 1$ , `safeAddr` is always inside the data region. An invalid address is therefore mapped into the data region instead of being rejected.

The masking also preserves the address modulo 4, so the alignment behaviour of the original access is preserved. The advantage of this approach is that it avoids the conditional bounds checks around each memory access.<sup>3</sup>

### 3.1.2 Implementation

The address masking is implemented in `RewriteMask.hs`. The `Rewrite.hs` program selects whether the baseline or masking rewriter is used. At the beginning of a program rewritten with address masking, we insert the following three instructions:

---

<sup>2</sup>*micro-eBPF and software fault isolation: Reference definition for mini-project 2*, Section 6.2, p. 9.

<sup>3</sup>Gang Tan, *Principles and Implementation Techniques of Software-Based Fault Isolation*, Section 3.1.3, pp. 152–153.

```

mov32 r11, r1
mov32 r12, r2
sub32 r12, 1

```

In a masking initial state, `r1` contains `DB`, while `r2` contains the size of the data region, `DL-DB`. After the prologue, we therefore have

$$r11 = DB$$

and

$$r12 = (DL - DB) - 1.$$

So `r11` stores the base address and `r12` stores the mask. We copy these values at the beginning because an input program is allowed to overwrite `r1` and `r2`, while registers `r11`–`r15` are reserved for code inserted by the rewriter.<sup>4</sup>

We use `r13` as a temporary register when rewriting memory accesses.<sup>5</sup> For example, a load without an offset is rewritten as:

```

mov32 r13, src
and32 r13, r12
add32 r13, r11
load32 dst, [r13]

```

First, the original address is copied into `r13`. We then apply the mask using `r12` and finally add `DB` using `r11`. The load is then performed using the new address in `r13`.

If the original memory access has an offset, we add the offset before applying the mask. We therefore first calculate

$$EA = src + offset$$

and then

$$safeAddr = DB + (EA \& mask).$$

This is necessary because the effective address of a micro-eBPF load or store includes its offset.<sup>6</sup> Stores are rewritten in the same way, so both loads and stores use the masked address instead of the original address.

The variation changes the contract with the runtime because the baseline initial states do not require the data-region size to be a power of two or `DB` to be aligned to that size. We therefore added `genMaskingInit`, which generates data regions whose size is a power of two, with  $m \geq 9$ , and where `DB` is aligned to the region size. We also added `randomMaskingStates`. The test kit uses these states when the `-masking` option is selected. Without this option, the original baseline initial-state generator is used.

---

<sup>4</sup>*micro-eBPF and software fault isolation: Reference definition for mini-project 2*, Sections 1.1 and 2.3, pp. 2 and 4.

<sup>5</sup>Gang Tan, *Principles and Implementation Techniques of Software-Based Fault Isolation*, Section 3.1.6, pp. 159–160.

<sup>6</sup>*micro-eBPF and software fault isolation: Reference definition for mini-project 2*, Table 3, p. 5.

### 3.1.3 Evaluation

We evaluated the address-masking implementation using both the supplied example programs and randomly generated programs. For the masking tests we used the `-masking` option, so the generated initial states satisfy the runtime requirements of the variation.

The implementation was tested on all 24 example programs. These examples include memory accesses with and without offsets, misaligned accesses, loops, overwritten registers, and jumps outside the code region. The rewritten programs produced no failed tests. Some executions of `scan_zero.asm` were inconclusive because the rewritten execution exhausted the available step budget, but none were reported as failures.

We also fuzz-tested the final implementation on 200 randomly generated well-formed programs using masking-compatible initial states. The result was:

200 programs, 0 failed.

The fuzz testing gives additional evidence that the rewriter handles a larger range of programs than the supplied examples.

The reason the masking enforces data-access integrity is that `r11` stores `DB` and `r12` stores `size-1`. Every rewritten memory access uses the address

$$DB + (EA \& (size - 1)).$$

Since `size` is a power of two, the masked part is always between 0 and `size - 1`. The resulting address is therefore always inside  $[DB, DL)$ . For an address that was already inside the data region, the alignment requirement on `DB` means that masking preserves the original address. Therefore valid memory accesses preserve their original address, while invalid addresses are mapped into the allowed region.<sup>7</sup>

Control-flow integrity is handled in the same way as in the baseline. Jumps whose original targets are inside the program are preserved through the symbolic labels used by the rewriter, while jumps whose targets are outside the program are redirected to `ErrorStub`. Address masking therefore changes the handling of memory accesses, but does not remove the control-flow protection.

The main trade-off is that invalid memory accesses are handled differently. The baseline redirects an out-of-bounds access to an error, while address masking maps the address into the allowed region and continues execution. For executions where the original program already violates the SFI policy, preserving the result is not required; the rewritten execution must instead remain within the SFI policy.<sup>8</sup>

As with the baseline evaluation, these tests provide evidence for the correctness of the implementation, but testing a finite number of programs and initial states is not a formal proof.

## 3.2 Extensions

### 64-Bit operations

Our current baseline is restricted to 32-bit operations. That is obvious by the likes of:

---

<sup>7</sup>*micro-eBPF and software fault isolation: Reference definition for mini-project 2*, Section 6.2, p. 9.

<sup>8</sup>*micro-eBPF and software fault isolation: Reference definition for mini-project 2*, Sections 4.1–4.2 and 6.2, pp. 7 and 9.

- Binary B32...
- Load B32...
- Store B32 ...

The main issue with this extension is that can no longer assume that every value and memory access fits into 32 bits.

The first change would be in the language itself. The syntax and semantics would have to support a B64 version of the relevant instructions.<sup>9</sup>

The semantics would also need to use 64-bit values for these operations instead of Word32. The memory model would also have to define what 64-bit load or store means.

In our rewriter we currently check the address:  $DB \leq address < DL$ . For a 32-bit access this is enough. For a 64-bit access, we need to make sure that the whole 8-byte access stays inside the data region. So conceptually:

- $address \geq DB$
- and
- $address + 8 \leq DL$

So an access close to DL could be valid for a 32-bit access but invalid for a 64-bit access. If we keep the original requirement that memory accesses must be aligned, a 64-bit access would normally need an appropriate alignment, for example an address divisible by 8.

To evaluate it we would add test programs containing:

- 64-bit arithmetic
- 64-bit loads and stores
- 64-bit values near the maximum value
- 64-bit accesses near DB and DL
- invalid 64-bit accesses
- alignment violations

Then just use the same approach as our baseline evaluation. Compare the original and rewritten programs over many initial states and check that the rewritten program preserves the behavior while preventing SFI violations.

### Unaligned memory access

Our baseline assumes that memory accesses are aligned to 4-byte boundaries. The semantics currently reject an access if the address is not divisible by 4. With unaligned accesses, this restriction would be removed.

However, the memory-safety check becomes slightly more complicated. With 32-bit access, we again check the starting address:  $DB \leq address < DL$ , but with unaligned accesses, a 4-byte access starting at address uses:

- address
- $address + 1$

---

<sup>9</sup>detailed-micro-eBPF-specification.pdf. p11 6.6

- $\text{address} + 2$
- $\text{address} + 3$

So we need to make sure that the entire accessed range is inside the data region. The check would need to ensure something like:  $DB \leq \text{address}$  and  $\text{address} + 3 < DL$ .

### Instructions that require more than one 64-bit word

This extension concerns the representation of instructions rather than the memory access itself. Our current instructions can be represented using a single 64-bit word. If we introduce instructions that require more than one word, the layout of the program becomes more complicated.

Our current **Layout.hs** helps by converting jump targets to labels and calculating the final offsets when assembling the program. To support multiword instructions, the layout system would need to account for the size of each instruction, rather than simply assuming that every instruction occupies one word.

The rewriter itself could still work in much the same way, but **Layout.hs** and the assembler would need to understand variable-size instructions.

## 3.3 Other possible variations

### Use/require special data regions such as power-of-two-base-addresses

The basic idea is using special bit patterns for addresses in the data region, to make address checks more efficient. Specifically this is done using the upper bits, which are referred to as the data region ID. Using this method, only one check is needed, as you only need to check whether a given address possess the right data region ID.<sup>10</sup>

### Use guard zones and the optimizations they enable

We place a guard zone directly before and after the data region. A guard zone is a region of memory just below and above the allowed data region that nothing is allowed to use, it is unmapped. Any access to the guard regions makes the hardware raise a trap which forces the program to stop.

An address is safe if it lies in the interval  $[DB - GSize, DL + GSize]$ , which is within the data region, plus the guard zones. This is considered safe because accessing an address in the guard zones stops the program, such that it cannot read or write anywhere it should not.

The idea is that with guard-zones, addresses only have to be near the data region, because an access outside, but close to the data region, lands in the guard zone.<sup>11</sup>

### Employ static value interval analysis to eliminate memory access checks

For every program point we compute a range a value can take at any run. If this range proves that all relevant addresses is inside the data region, the program is considered safe, and the basic runtime checks become unnecessary.

---

<sup>10</sup>SFI-slides p. 25-27

<sup>11</sup>Tan-SFI-principles.pdf. p154-158